

Episode 5: Your First Network CI/CD Pipeline

Video Title

"Network as Code: GitOps for Your Network"

Target Length

20-25 minutes

Goal

Show the full CI/CD workflow: push triggers validation, PR shows a config diff, merge deploys to devices. Make the viewer understand that this is the same workflow software engineers use, applied to networking. Do not walk through the GitHub Actions YAML in detail.

Pre-Recording Checklist

Before you hit record, make sure:

- GitHub repo is up to date with all labs
 - ContainerLab topology is running (all 6 devices UP)
 - Have a feature branch ready to demonstrate (or create one live)
 - GitHub Actions workflows exist in `.github/workflows/`
 - Know which data change to make for the demo (something small and safe)
 - Have the GitHub PR page ready to show the diff comment
-

SEGMENT 1: Recap and the Problem (0:00 - 2:30)

What to show: Nothing on screen, or a simple diagram of the traditional workflow.

Talking points:

- "We have a data model, validation, and config generation. But right now, that entire workflow lives on my laptop. I run validate, I run render, I run deploy. If I forget a step, or skip validation because I am in a hurry, nothing stops me."
 - "Lab 4 fixes that by putting the entire workflow into a CI/CD pipeline. Push code, automation runs. No shortcuts, no skipped steps, no deploying without validation."
 - "If you work in software engineering, this is standard practice. Every code change goes through a pipeline. Network engineering has been slow to adopt this, but the concepts are identical."
-

SEGMENT 2: The Three Pipeline Stages (2:30 - 5:00)

What to show: A simple diagram or just describe verbally.

Talking points:

- "The pipeline has three stages, triggered by three different Git events."
 - "Stage one: you push to a feature branch. The pipeline runs all four validation layers. Format, syntax, semantic, compliance. If any of them fail, you know immediately. This is the 'does my change make sense' check."
 - "Stage two: you open a pull request. The pipeline runs validation again, then generates the configs and posts a diff as a PR comment. You and your reviewer can see exactly what will change on each device before approving. This is the 'what will this change do' check."
 - "Stage three: you merge to main. The pipeline deploys the configs to the devices via Scrapli. After deployment, it runs the post-change tests from Lab 5. This is the 'did it work' check."
 - "Every stage has a gate. Validation fails, the PR is blocked. Reviewer does not approve, the merge is blocked. Post-change tests fail, you get an alert. At no point can a misconfiguration silently reach the network."
-

SEGMENT 3: The Workflow Files (5:00 - 7:00)

What to show: List the workflow files. Do NOT show their contents in detail.

```
ls .github/workflows/
```

Talking points:

- "Four workflow files. validate.yaml runs on every push to a feature branch. plan.yaml runs when a PR is opened or updated. deploy.yaml runs when code merges to main. drift_check.yaml runs on a schedule, but that is Lab 6."
 - "I am not going to walk through the YAML line by line. GitHub Actions syntax is well-documented and the build guide covers every step. What matters is the concept: each file maps to one stage of the pipeline."
-

SEGMENT 4: Live Demo - Feature Branch Push (7:00 - 12:00)

What to show: Make a change, push it, watch the pipeline run.

Step 1: Create a feature branch

```
git checkout -b feature/demo-vrf-change
```

Step 2: Make a small data model change

Make a simple, safe edit. Add a description to a VRF, or change an interface description. Show only the line you are editing.

- "I am making a small change. Nothing dramatic. Just enough to trigger the pipeline."

Step 3: Commit and push

```
git add data/services/vrfs.yaml
```

```
git commit -m "Update VRF description for demo"
git push -u origin feature/demo-vrf-change
```

Step 4: Show GitHub Actions running

Switch to the browser and show the Actions tab.

- "There it is. The validate workflow kicked off automatically. It is running all four validation layers against the data model."

Wait for it to complete.

- "All checks passed. Green across the board. If I had broken something in the YAML, this would be red and I would know before anyone else even looked at the change."

SEGMENT 5: Live Demo - Pull Request (12:00 - 17:00)

What to show: Open a PR, watch the plan workflow run, show the diff comment.

Step 1: Open the PR

```
gh pr create --title "Update VRF description" --body "Demo of the CI/CD pipeline"
```

Or do it in the browser.

Step 2: Show the plan workflow

- "A second workflow triggered. The plan workflow runs validation again, then generates configs from both the main branch and this branch, and diffs them."

Wait for it to complete.

Step 3: Show the PR comment

Navigate to the PR and show the automatically posted comment.

Talking points:

- "Look at this. The pipeline generated a config diff and posted it right here on the PR. You can see exactly which devices are affected and what lines changed."
- "This is the network equivalent of a Terraform plan. Before you approve this change, you know exactly what it will do. No surprises."
- "Your reviewer does not need to understand the YAML data model in depth. They can read the actual config diff and make an informed decision."
- "Compare this to the traditional workflow. Someone sends an email saying 'I am going to change the VRF config on the leafs.' What does that actually mean? Which lines? Which devices? With the pipeline, those questions answer themselves."

SEGMENT 6: Live Demo - Merge and Deploy (17:00 - 21:00)

What to show: Merge the PR, watch deployment happen.

Step 1: Merge

Merge the PR in the browser or via CLI:

```
gh pr merge --squash
```

Step 2: Show the deploy workflow

- "The deploy workflow just triggered. It is generating the final configs from the merged main branch and pushing them to the devices via Scrapli."

Step 3: Show the deployment completing

Wait for the workflow to finish.

Talking points:

- "Deployment complete. The pipeline connected to all six devices over SSH using Scrapli, pushed the updated configs, and verified they applied cleanly."
- "After deployment, the post-change tests run automatically. We will cover those in detail next episode, but you can see them running here as the final step."
- "If the tests fail, the pipeline flags it immediately. You know within minutes whether your change worked, not hours or days later when a user reports a problem."

Step 4: Verify on the device (optional)

```
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "show running-config" | grep -A2 "vrf"
```

- "And there it is on the device. The change we made in YAML, validated by the pipeline, approved in a PR, and deployed automatically."

SEGMENT 7: Branching Strategy (21:00 - 22:30)

What to show: Simple slide or just talking points.

Talking points:

- "A quick note on branching strategy. Main is your production branch. It is protected. Nobody pushes directly to main."
- "Feature branches are where changes happen. feature/add-vrf-production, feature/update-ospf-timers. Each one gets a PR, a review, and a pipeline run."
- "Hotfix branches exist for emergencies. If something is broken in production and you need to fix it fast, a hotfix branch has an expedited review process. But it still goes through the pipeline. No cowboy commits to main."
- "This is standard GitOps. Software teams have been doing this for years. The only difference is that the 'application' is your network."

SEGMENT 8: The Close (22:30 - 24:00)

What to show: Series overview or back to camera.

Talking points:

- "That is Lab 4. A full CI/CD pipeline for network changes. Push triggers validation, PRs show config diffs, merges deploy to devices."
 - "The build guide walks through setting up every workflow file, configuring the Scrapli deployment script, and wiring up notifications. Available at [your link] for \$49."
 - "Next episode, we talk about what happens after deployment. How do you know your change actually worked? 31 pytest tests that verify OSPF neighbors, BGP sessions, route tables, and full-mesh reachability. That is Lab 5."
 - "Subscribe, and I will see you in the lab."
-

Commands Reference (Quick Copy)

```
# List workflow files
ls .github/workflows/

# Feature branch workflow
git checkout -b feature/demo-change
git add data/services/vrfs.yaml
git commit -m "Update VRF description"
git push -u origin feature/demo-change

# Open PR
gh pr create --title "Update VRF description" --body "Demo"

# Merge PR
gh pr merge --squash

# Check device after deploy
docker exec clab-nac-spine-leaf-leaf1 vtysh -c "show running-config" | grep -A2 "vrf"
```

Do NOT Show On Camera

- GitHub Actions workflow YAML in detail (paid content)
- The Scrapli deploy script internals (paid content)
- How to set up GitHub Actions secrets and runners (paid content)
- The step-by-step pipeline build process (paid content)

DO Show On Camera

- The workflow file names (ls output)
- GitHub Actions UI showing pipeline runs (pass/fail)
- The auto-generated PR comment with config diffs
- The feature branch workflow (push, PR, merge) in action
- Device verification after deployment
- The branching strategy concept